



Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

Bifurcations and chaos - Lab examples

CSC1111
Building Complex Computational
Models

Marija Bezbradica

Example 11.1 (Sayama book)

For this example we will use Sayama book and *pycxsimulator.py* for Bifurcation simulations (introduced in weeks 6-7 lectures and slides <https://github.com/hsayama/PyCX>)

Three essential components of computer simulation

Initialize. You will need to set up the initial values for all the state variables of the system.

Observe. You will need to define how you are going to monitor the state of the system. This could be done by just printing out some variables, collecting measurements in a list structure, or visualizing the state of the system.

Update. You will need to define how you are going to update the values of those state variables in every time step. This part will be defined as a function, and it will be executed repeatedly.

Example 11.1 (cont.)

Let's implement a simple linear difference equation model of a single variable (growth/decay model):

$$x_t = ax_{t-1}$$

Using the three stages described above, the code might look something like this:

```
from pylab import *
```

```
a = 1.1
```

```
def initialize():
```

```
    global x, result
```

```
    x = 1.
```

```
    result = [x]
```

```
def observe():
```

```
    global x, result
```

```
    result.append(x)
```

```
def update():
```

```
    global x, result
```

```
    x = a * x
```

```
initialize()
```

```
for t in range(30):
```

```
    update()
```

```
    observe()
```

```
plot(result)
```

```
show()
```

Example 11.1 (cont.)

Exercise 11.2 - Conduct simulations of the previous model for various values of parameter a (e.g. from 0.0 to 3.0) to see what kind of behaviors are possible in this model and how the value of a determines the resulting behavior.

Exercise 11.3 Implement a simulation code of the following difference equation:

$$x_t = ax_{t-1} + b, \quad x_0 = 1$$

This equation is still linear, but now it has a constant term in addition. Some real-world examples that can be modeled in this equation include fish population growth with constant removals due to fishing, and growth of credit card balances with constant monthly payments (both with negative b). Change the values of a and b , and see if the system's behaviors are the same as those of the simple exponential growth/decay model.

Example 11.4 (Sayama book)

Simulating discrete time models with multiple variables – In this model, we are looking at a discrete time system described by two variables x and y :

$$x_t = 0.5x_{t-1} + y_{t-1}$$

$$y_t = -0.5x_{t-1} + y_{t-1}$$

$$x_0 = 1, \quad y_0 = 1$$

$$\begin{pmatrix} x \\ y \end{pmatrix}_t = \begin{pmatrix} 0.5 & 1 \\ -0.5 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}_{t-1}$$

Try implementing the simulation code of the equation using the the **intialize()** - **observe()** - **update()** functions of the **PyCX** simulator.

Example 11.4 (cont.)

Your new code may look like this:

```
from pylab import *
```

```
def initialize():
```

```
    global x, y, xresult, yresult
    x = 1.
    y = 1.
    xresult = [x]
    yresult = [y]
```

```
def observe():
```

```
    global x, y, xresult, yresult
    xresult.append(x)
    yresult.append(y)
```

```
def update():
```

```
    global x, y, xresult, yresult
    nextx = 0.5 * x + y # required for sim. update
    nexty = -0.5 * x + y # required for sim. update
    x, y = nextx, nexty
```

```
initialize() for t in xrange(30):
```

```
    update()
    observe()
```

```
plot(xresult, 'b-')
plot(yresult, 'g--')
show()
```

Example 11.5 (cont.)

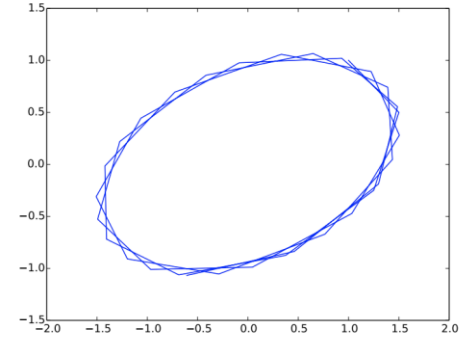
There is another way of visualising simulation results in a *phase space*:

Replace the following part in the previous code:

```
plot(xresult)
plot(yresult)
```

with this:

```
plot(xresult, yresult)
```



This will produce a trajectory of the system state in an x-y phase space, which clearly shows that the system is in an oval, periodic oscillation, which is a typical signature of a linear system.

Exercise: Simulate the above two-variable system using a range of different coefficients in the equations and see what kind of behaviors can arise. E.g. try values for ± 0.5 , ± 0.1 , ± 0.51 , ± 0.6

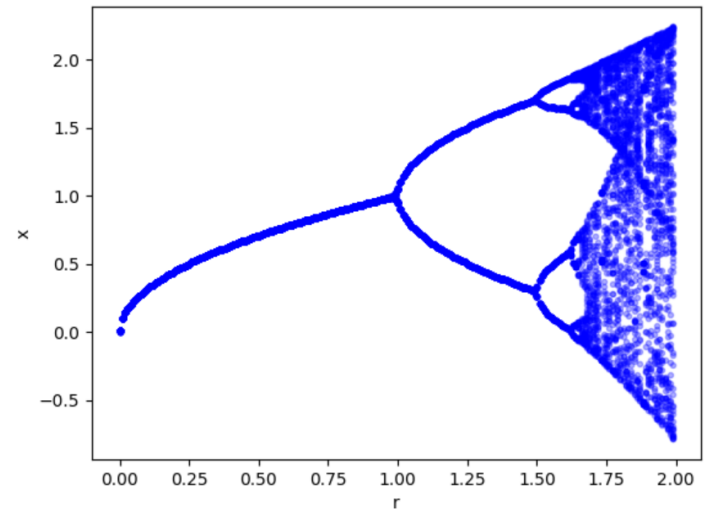
- Note: The full solution has been uploaded on your Loop, week 11, 'ds-oscillation-correct-phasespace.py'

Example 11.6

Draw a bifurcation diagram for:

$$x_t = x_{t-1} + r - x_{t-1}^2$$

The output looks like this:



Consulting the lecture notes, think about all the critical values in this diagram and discuss changes in the system (can you e.g. notice where is chaos?)

- Note: The full code has been uploaded on your Loop, week 11, 'ds-bifurcation-diagram-numerical.py'
- Check more on bifurcation and chaos regions on:

<https://www.geogebra.org/m/wQbHRgye>

Example 11.7

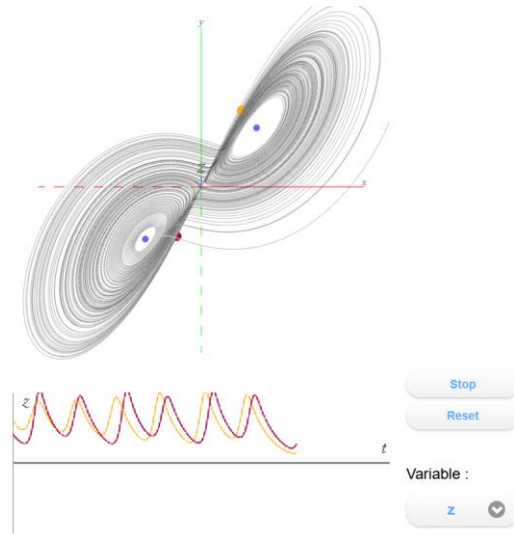
Check Lorenz Attractor plots and read how you can distinguish different regions and attractors of interest:

<https://www.t-ott.dev/2022/02/24/lorenz-attractor>

For 3D plot for Lorenz eq check. You can flip diagrams around to see how the two points are :

<https://experiences.mathemarium.fr/Lorenz-attractor>

You can see what is happening for the two strange attractors how they have a very fine response to the initial conditions:



Exercise 11.7 (FYI only) Fibonacci examples

Really interesting description of the origins, justification and (many) applications of the sequence

<https://r-knott.surrey.ac.uk/Fibonacci/fibnat.html>